

Prévision du niveau du réservoir WADI

Rapport de projet — Master IA

Mohammad Diab
Université Clermont Auvergne

3 septembre 2026

Table des matières

1	Introduction	3
1.1	Problématique	3
1.2	Objectif	3
1.3	La variable cible (target)	3
1.4	Le dataset	3
2	Environnement de travail	3
2.1	Environnement virtuel (venv)	4
2.2	Le kernel Jupyter	4
3	Type de problème : forecasting multivarié en régression	4
3.1	Forecasting (prévision)	4
3.2	Multivarié	4
3.3	Régression	4
3.4	Schéma du problème	5
4	Les trois modèles testés	5
4.1	Cadre mathématique commun	5
4.2	Persistence (baseline)	5
4.3	Random Forest	6
4.4	XGBoost	6
4.5	Comparaison mathématique des trois modèles	7
4.6	Synthèse : le rôle principal commun	8
5	Méthodologie	8
5.1	Prétraitement des données	8
5.2	Analyse univariée	8
5.3	Analyse bivariée et corrélation	8
5.3.1	Mathématiques de la corrélation de Pearson	9
5.4	Feature engineering	9
5.5	Découpage train / test	10
5.6	Modélisation	10
5.7	Évaluation	10

5.7.1	Formules des métriques	11
6	Conclusion	11

1 Introduction

1.1 Problématique

WADI est un banc d’essai réel de distribution d’eau. Chaque minute, le système SCADA (*Supervisory Control And Data Acquisition*) enregistre de nombreuses mesures : niveau des réservoirs, débits, état des pompes et des vannes, pression, etc.

Dans ce projet, on se pose une question simple mais importante : **peut-on prédire à l’avance le niveau du réservoir principal ?** Si oui, on pourrait anticiper un débordement ou un niveau trop bas, et agir avant que le problème ne devienne grave.

1.2 Objectif

L’objectif principal est de prédire le niveau du réservoir principal 2_LT_001_PV à un horizon de **10 minutes**, en utilisant les 129 variables SCADA enregistrées chaque minute sur 14 jours.

Concrètement, on veut :

- comprendre les données WADI ;
- identifier les variables utiles pour la prédiction ;
- comparer trois modèles : Persistence, Random Forest et XGBoost ;
- battre la baseline Persistence d’environ **20 %** en RMSE.

1.3 La variable cible (target)

La variable qu’on cherche à prédire s’appelle la **target**. Ici, c’est 2_LT_001_PV, qui représente le niveau du réservoir principal en zone 2, exprimé en pourcentage (%).

Les noms des colonnes suivent une convention SCADA. Pour 2_LT_001_PV :

Partie	Signification
2	Zone du procédé (zone 2)
LT	<i>Level Transmitter</i> — transmetteur de niveau
001	Numéro du capteur
PV	<i>Process Value</i> — valeur mesurée

1.4 Le dataset

Propriété	Valeur
Fichier	WADI_14days_new.csv
Durée	14 jours
Fréquence	1 mesure par minute
Nombre de lignes	~784 570
Nombre de variables	129 (+ Date, Time, Row)
Target	2_LT_001_PV

2 Environnement de travail

Avant de travailler sur le notebook, on a mis en place un environnement de développement propre. Voici ce qu’on a appris en le faisant.

2.1 Environnement virtuel (venv)

Un **environnement virtuel** (venv) est un dossier isolé qui contient sa propre version de Python et ses propres bibliothèques (pandas, scikit-learn, etc.).

Pourquoi on l'utilise ?

- Pour ne pas mélanger les paquets de différents projets ;
- Pour éviter les conflits de versions entre bibliothèques ;
- Pour que le projet soit reproductible : quelqu'un d'autre peut recréer le même environnement avec `requirements.txt`.

On a créé l'environnement avec :

```
cd my_wadi
python3 -m venv .venv
source .venv/bin/activate    # Linux / macOS
pip install -r requirements.txt
```

2.2 Le kernel Jupyter

Quand on ouvre un notebook Jupyter (.ipynb), le code ne s'exécute pas tout seul : il a besoin d'un **kernel**.

Le kernel, c'est le **moteur Python** qui fait tourner les cellules du notebook. Chaque fois qu'on clique sur "Exécuter", c'est le kernel qui calcule le résultat.

En résumé :

- **venv** = l'environnement isolé avec les bonnes bibliothèques ;
- **kernel** = le moteur qui exécute le code du notebook dans cet environnement.

3 Type de problème : forecasting multivarié en régression

Notre problème est un **forecasting multivarié temporel en régression**. Décortiquons chaque mot.

3.1 Forecasting (prévision)

On veut prédire une valeur **dans le futur** (dans 10 minutes) à partir de données **passées**. C'est de la prévision temporelle.

3.2 Multivarié

On n'utilise pas seulement le niveau du réservoir. On utilise **plusieurs variables en même temps** : débit, pression, état de la pompe, etc. C'est ce qu'on appelle *multivarié*.

3.3 Régression

C'est le point clé : on fait de la **régression**, pas de la classification.

Pourquoi régression ? Parce que toutes les valeurs du dataset qu'on utilise pour modéliser sont **numériques** :

- le niveau du réservoir est un nombre continu (ex. 73,5 %) ;
- les débits, pressions et états des pompes sont aussi des nombres ;
- la target qu'on prédit est un pourcentage, donc un nombre réel.

En **classification**, on prédit une **catégorie** (ex. “pompe ON” ou “pompe OFF” comme étiquette). Ici, on prédit un **nombre continu** : le niveau exact dans 10 minutes. C’est donc un problème de **régression**.

3.4 Schéma du problème

À l’instant t , on lit les features (variables + leurs lags). On veut prédire la valeur de 2_LT_001_PV à l’instant $t + 10$ minutes.

4 Les trois modèles testés

On a choisi trois modèles. Chacun a un rôle différent, mais tous font la même chose au fond : de la **régression** pour prédire le niveau futur.

4.1 Cadre mathématique commun

Pour chaque minute t , on dispose d’un vecteur de features $\mathbf{x}_t \in \mathbb{R}^p$ (variables SCADA + leurs lags). La target à prédire est le niveau du réservoir à l’horizon $H = 10$ minutes :

$$y_t = 2_LT_001_PV(t + H).$$

Le problème de régression consiste à trouver une fonction f telle que :

$$\hat{y}_t = f(\mathbf{x}_t) \approx y_t. \quad (1)$$

L’objectif d’entraînement est de minimiser une fonction de perte sur le jeu d’entraînement. En régression, on utilise souvent l’erreur quadratique moyenne (MSE) :

$$\mathcal{L}(f) = \frac{1}{n_{\text{train}}} \sum_{t=1}^{n_{\text{train}}} (y_t - f(\mathbf{x}_t))^2. \quad (2)$$

Les trois modèles diffèrent par la **forme** de la fonction f qu’ils apprennent.

4.2 Persistence (baseline)

Type : baseline (référence minimale).

Définition mathématique. Persistence suppose que le niveau futur est égal au niveau actuel. Avec $H = 10$ minutes :

$$\hat{y}_t^{\text{persist}} = y_t^{\text{niveau}} = 2_LT_001_PV(t). \quad (3)$$

Autrement dit, f est la fonction identité sur la seule variable de niveau à l’instant t :

$$f_{\text{persist}}(\mathbf{x}_t) = x_t^{(\text{LT})},$$

où $x_t^{(\text{LT})}$ est la composante du vecteur $\mathbf{x}_t$ correspondant au niveau actuel du réservoir.

Erreur associée. Pour une observation, l'erreur de Persistence est simplement :

$$e_t = y_t - \hat{y}_t^{\text{persist}} = 2_LT_001_PV(t + H) - 2_LT_001_PV(t).$$

C'est la variation réelle du niveau sur 10 minutes. Si le niveau change peu, Persistence est déjà performante.

Rôle : c'est le plancher de performance. Si un modèle complexe ne bat pas Persistence, il ne sert à rien. C'est notre point de comparaison.

4.3 Random Forest

Type : Machine Learning — forêt d'arbres de décision (bagging).

Entrée : toutes les variables prioritaires avec leurs lags (1, 2, 5, 10 min).

Étape 1 — Un arbre de décision. Un arbre partitionne l'espace des features en régions rectangulaires. À chaque nœud, on choisit une variable $x^{(j)}$ et un seuil s pour diviser les données :

$$\text{gauche : } x^{(j)} \leq s \quad \text{vs} \quad \text{droite : } x^{(j)} > s.$$

En régression, le critère de division minimise la variance (ou MSE) dans chaque feuille. Pour un ensemble d'indices I dans un nœud, la MSE est :

$$\text{MSE}(I) = \frac{1}{|I|} \sum_{t \in I} (y_t - \bar{y}_I)^2, \quad \bar{y}_I = \frac{1}{|I|} \sum_{t \in I} y_t. \quad (4)$$

La prédiction d'une feuille L est la moyenne des targets qu'elle contient : $\hat{y}_L = \bar{y}_L$.

Étape 2 — Bagging (Bootstrap Aggregating). Random Forest construit $B = 100$ arbres (paramètre `n_estimators=100` dans notre notebook). Pour chaque arbre $b = 1, \dots, B$:

1. on tire un échantillon bootstrap $\mathcal{D}_b$ (tirage avec remise) ;
2. on entraîne un arbre T_b sur $\mathcal{D}_b$;
3. à chaque split, on ne considère qu'un sous-ensemble aléatoire de features (réduit la corrélation entre arbres).

Prédiction finale — moyenne des arbres :

$$\hat{y}_t^{\text{RF}} = f_{\text{RF}}(\mathbf{x}_t) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}_t). \quad (5)$$

Pourquoi ça marche : chaque arbre capture des règles **non-linéaires** (ex. “si la pompe est ON et le débit $> 1,5$, alors le niveau monte”). La moyenne de nombreux arbres réduit la variance et rend le modèle plus stable.

Rôle : premier vrai modèle ML, robuste et interprétable via la *feature importance* (fréquence d'utilisation de chaque variable dans les splits).

4.4 XGBoost

Type : Machine Learning — gradient boosting d'arbres.

Entrée : mêmes features que Random Forest.

Idée générale. Contrairement à Random Forest (arbres en parallèle), XGBoost construit les arbres **séquentiellement**. Chaque nouvel arbre corrige les erreurs du modèle précédent.

Modèle additif. Après m itérations, la prédiction s'écrit :

$$\hat{y}_t^{(m)} = \hat{y}_t^{(m-1)} + \eta \cdot h_m(\mathbf{x}_t), \quad (6)$$

où h_m est le m -ième arbre et $\eta \in (0, 1]$ est le *learning rate* (taux d'apprentissage).

Fonction objectif. XGBoost minimise une perte + une pénalisation de complexité :

$$\mathcal{J}^{(m)} = \sum_{t=1}^n \ell(y_t, \hat{y}_t^{(m-1)} + h_m(\mathbf{x}_t)) + \Omega(h_m), \quad (7)$$

où ℓ est la fonction de perte (souvent la MSE en régression) et Ω pénalise la complexité de l'arbre (nombre de feuilles, magnitude des scores).

Approximation par développement de Taylor. Pour trouver le meilleur arbre h_m , XGBoost utilise une approximation du second ordre de la perte autour de $\hat{y}_t^{(m-1)}$. On définit les résidus (gradients) :

$$g_t = \left. \frac{\partial \ell(y_t, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=\hat{y}_t^{(m-1)}}, \quad h_t = \left. \frac{\partial^2 \ell(y_t, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=\hat{y}_t^{(m-1)}}. \quad (8)$$

Avec la MSE, $\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$, on obtient simplement :

$$g_t = \hat{y}_t^{(m-1)} - y_t \quad (\text{erreur actuelle du modèle}),$$

$$h_t = 1.$$

Le nouvel arbre h_m est alors construit pour prédire ces résidus g_t : il apprend à corriger les erreurs restantes.

Prédiction finale. Après $M = 100$ arbres (`n_estimators=100`) :

$$\hat{y}_t^{\text{XGB}} = f_{\text{XGB}}(\mathbf{x}_t) = \sum_{m=1}^M \eta \cdot h_m(\mathbf{x}_t). \quad (9)$$

Rôle : modèle plus performant que Random Forest sur données tabulaires, car il affine progressivement les prédictions. mais risque un *overfitting* (surapprentissage) si M est trop grand ou si η est trop élevé.

4.5 Comparaison mathématique des trois modèles

Modèle	Forme de f	Entraînement	Complexité
Persistence	$f(\mathbf{x}_t) = x_t^{(\text{LT})}$	Aucun (règle fixe)	$O(1)$
Random Forest	$\frac{1}{B} \sum_b T_b(\mathbf{x}_t)$	B arbres en parallèle (bagging)	Moyenne
XGBoost	$\sum_m \eta \cdot h_m(\mathbf{x}_t)$	M arbres séquentiels (boosting)	Élevée

4.6 Synthèse : le rôle principal commun

Les trois modèles ont le **même rôle principal** : faire de la régression pour prédire 2_LT_001_PV à $t + 10$ minutes.

Modèle	Rôle spécifique	Complexité
Persistence	Référence minimale à battre	Très faible
Random Forest	Premier modèle ML robuste	Moyenne
XGBoost	Amélioration par boosting	Élevée

La différence n'est pas *ce qu'on prédit* (toujours le niveau), mais *comment* on l'apprend.

5 Méthodologie

Cette section décrit chaque étape du notebook et explique **pourquoi** on la fait.

5.1 Prétraitement des données

Ce qu'on fait :

1. Trier les données par ordre chronologique.
2. Imputer les valeurs manquantes par la **médiane** sur les colonnes numériques.
3. Ne pas supprimer de lignes (on garde l'ordre temporel intact).

Pourquoi (le rôle) :

- Le tri chronologique est **indispensable** en forecasting : le temps a un sens.
- On utilise la **médiane** et non la moyenne car elle est robuste aux valeurs extrêmes (un pic de débit ne va pas fausser toute la colonne).
- On évite de supprimer des lignes car cela créerait des “trous” dans la série temporelle.

5.2 Analyse univariée

Ce qu'on fait :

- Histogramme, densité (KDE) et boxplot de la target 2_LT_001_PV.
- Série temporelle sur 3 jours pour voir la dynamique du niveau.
- Même analyse pour 3 variables clés : débit (2_FIT_001_PV), pompe (2_P_003_STATUS), pression (2_PIT_001_PV).

Pourquoi (le rôle) : Avant de modéliser, il faut **comprendre les données**. Un débit mesuré entre 0 et 2 n'a pas la même échelle qu'un niveau en %. L'analyse univariée nous permet de voir l'échelle, la forme et les tendances de chaque variable. On n'entraîne pas encore de modèle — on **observe** seulement.

5.3 Analyse bivariée et corrélation

Ce qu'on fait :

- Calculer la corrélation de Pearson entre 2_LT_001_PV et chaque colonne numérique.
- Afficher les 20 variables les plus corrélées (positivement ou négativement).
- Tracer des scatter plots des 2 variables les plus corrélées vs la target.

- Boxplot du niveau selon l'état de la pompe (ON/OFF).

Pourquoi (le rôle) : La corrélation est un **premier filtre** pour identifier les variables qui “bougent avec” le niveau du réservoir. Si deux variables montent ensemble, elles sont probablement utiles pour la prédiction.

On retient les variables avec $|r| > 0,3$ comme **prioritaires** pour le feature engineering. Ce seuil est un compromis :

- trop bas (ex. 0,1) : on garde trop de bruit ;
- trop haut (ex. 0,7) : on risque de perdre des variables utiles.

Attention : corrélation $\neq$ causalité. Deux variables peuvent être corrélées parce qu'elles réagissent toutes les deux à un troisième facteur (ex. une pompe qui s'allume).

5.3.1 Mathématiques de la corrélation de Pearson

La corrélation de Pearson mesure la **relation linéaire** entre deux variables X et Y . Sa formule est :

$$r_{X,Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (10)$$

où :

- n = nombre d'observations (minutes) ;
- x_i, y_i = valeurs des variables X et Y à l'instant i ;
- $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ = moyenne de X ;
- $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ = moyenne de Y .

Comment lire la formule :

1. Le **numérateur** $\sum (x_i - \bar{x})(y_i - \bar{y})$ mesure si X et Y varient dans le même sens (covariance).
2. Les **dénominateurs** normalisent ce score entre -1 et $+1$.
3. Si $r \approx +1$: quand X monte, Y monte aussi.
4. Si $r \approx -1$: quand X monte, Y baisse.
5. Si $r \approx 0$: pas de relation linéaire claire.

5.4 Feature engineering

Ce qu'on fait :

1. Sélectionner les **PRIORITY_FEATURES** = variables avec $|r| > 0,3$ + la target.
2. Créer des **lags** (1, 2, 5, 10 minutes) sur chaque variable prioritaire.
3. Créer la **target décalée** : **target** = niveau à $t + 10$ min (**shift(-10)**).
4. Supprimer les lignes avec NaN (début = pas assez de lag, fin = pas de target future).

Pourquoi (le rôle) : Les modèles tabulaires (Random Forest, XGBoost) ne “voient” qu'**une ligne à la fois**. Les lags leur donnent un **historique récent** : le modèle sait ce qu'était le niveau il y a 1, 2, 5 et 10 minutes. Comme notre horizon de prédiction est 10 minutes, le lag 10 est particulièrement pertinent.

Instant	Contenu	Rôle
t	Lags (niveau, débit, pompe à $t-1$, $t-2$, $t-5$, $t-10$)	Entrées (features)
$t + 10$	Valeur de 2_LT_001_PV dans <code>target</code>	Sortie à prédire

En entraînement, on connaît la target future grâce au dataset historique. En production, la target est inconnue — c’est précisément ce que le modèle doit prédire.

5.5 Découpage train / test

Ce qu’on fait :

- Découpage **temporel** : 80 % train / 20 % test.
- **Pas de mélange aléatoire** (pas de `shuffle`).

Pourquoi (le rôle) : En forecasting, mélanger les données aléatoirement reviendrait à **entraîner sur le futur et tester sur le passé** — ce qui triche. Le split temporel simule la réalité : on entraîne sur les ~ 11 premiers jours, on teste sur les ~ 3 derniers.

C’est comme si on disait au modèle : “voici ce qui s’est passé avant, maintenant prédit ce qui va arriver après”. Si on mélangeait, le modèle verrait des données du futur pendant l’entraînement et ses scores seraient artificiellement bons.

5.6 Modélisation

Ce qu’on fait : On entraîne les trois modèles sur le jeu d’entraînement (80 %) et on évalue sur le jeu de test (20 %).

Modèle	Variables utilisées	Logique
Persistence	2_LT_001_PV à l’instant t	Niveau actuel = prédiction
Random Forest	Toutes les <code>PRIORITY_FEATURES</code> lagguées	Apprend les splits utiles
XGBoost	Idem Random Forest	Corrige les erreurs séquentiellement

Pourquoi (le rôle) : On commence toujours par Persistence pour avoir une baseline. Ensuite Random Forest comme premier modèle ML, puis XGBoost pour voir si la complexité supplémentaire apporte un gain réel.

5.7 Évaluation

Ce qu’on fait :

- Tableau comparatif MAE / RMSE pour les 3 modèles.
- Tableau de prédictions minute par minute (réel vs chaque modèle).
- Scatter plot : prédiction vs vérité.
- Courbe temporelle sur 24 h : réel vs Persistence vs meilleur modèle.

Pourquoi (le rôle) : Les métriques donnent un score global, mais les graphiques montrent **quand** le modèle se trompe (ex. quand une pompe démarre). Un bon modèle doit coller à la courbe réelle dans le temps, pas seulement avoir un RMSE bas en moyenne.

5.7.1 Formules des métriques

MAE (Mean Absolute Error) — Erreur absolue moyenne

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (11)$$

où y_i = valeur réelle, $\hat{y}_i$ = valeur prédite, n = nombre d'observations.

Interprétation : si MAE = 2, en moyenne le modèle se trompe de 2 points de pourcentage. Plus c'est petit, mieux c'est.

RMSE (Root Mean Squared Error) — Racine de l'erreur quadratique moyenne

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (12)$$

Interprétation : comme la MAE, mais les erreurs sont **mises au carré** avant d'être moyennées. Résultat : le RMSE **pénalise davantage les grosses erreurs**. Si le modèle fait une grosse erreur sur un pic de niveau, le RMSE le remarquera plus que la MAE.

Amélioration par rapport à Persistence

$$\text{Amélioration (\%)} = \left(1 - \frac{\text{RMSE}_{\text{modèle}}}{\text{RMSE}_{\text{persistence}}} \right) \times 100 \quad (13)$$

Objectif du projet : obtenir une amélioration **supérieure à 20 %**. Par exemple, si Persistence a un RMSE de 5 et notre modèle un RMSE de 3,5 :

$$\text{Amélioration} = \left(1 - \frac{3,5}{5} \right) \times 100 = 30 \%$$

Le modèle bat Persistence de 30 %, ce qui dépasse l'objectif de 20 %.

MAE vs RMSE : quand utiliser quoi ?

Métrique	Avantage	Limite
MAE	Facile à interpréter (erreur typique en %)	Traite toutes les erreurs pareil
RMSE	Pénalise les grosses erreurs	Moins intuitif

Dans notre projet, on utilise surtout le **RMSE** pour comparer les modèles et mesurer l'amélioration vs Persistence.

6 Conclusion

Ce projet nous a permis de construire un pipeline complet de forecasting : chargement des données → analyse → feature engineering → modélisation → évaluation.

Ce qu'on a appris :

- La différence entre régression et classification (on prédit un nombre, pas une catégorie).
- L'importance du split temporel en forecasting.
- Le rôle des lags pour donner de la mémoire aux modèles tabulaires.
- Pourquoi on commence toujours par une baseline simple (Persistence).
- Comment interpréter MAE et RMSE pour juger un modèle.

Limites connues :

- Dataset court (14 jours seulement) : la généralisation est limitée.
- Peu d'événements rares (pannes) : le modèle peut les rater.

Pistes d'amélioration :

- Optimisation des hyperparamètres (GridSearch).
- Tester d'autres horizons (5 et 30 minutes).
- Ajouter des modèles de deep learning (LSTM, GRU, Transformer).
- Utiliser SHAP pour expliquer les prédictions.

Ce projet reste avant tout un **projet d'apprentissage** : l'objectif n'est pas seulement d'obtenir le meilleur score, mais de **comprendre** chaque étape du pipeline et chaque modèle testé.